

Model View Presenter (MVP) Design

AI-Powered Exam Preparation System

Module 1 — Document Upload & RAG Ingestion Pipeline

1. Introduction

This document explains the design of the initial core module for the AI-Powered Exam Preparation System. The focus of this phase is only on the document ingestion and retrieval pipeline.

The module allows users to:

- Upload PDF documents
- Parse document content
- Generate chunks
- Create embeddings
- Store vectors in a vector database
- Perform semantic retrieval

This module becomes the foundation of the complete Retrieval-Augmented Generation (RAG) architecture.

2. Scope of the First MVP

The first implementation phase intentionally avoids complex frontend and distributed architecture.

Only the following functionalities are included:

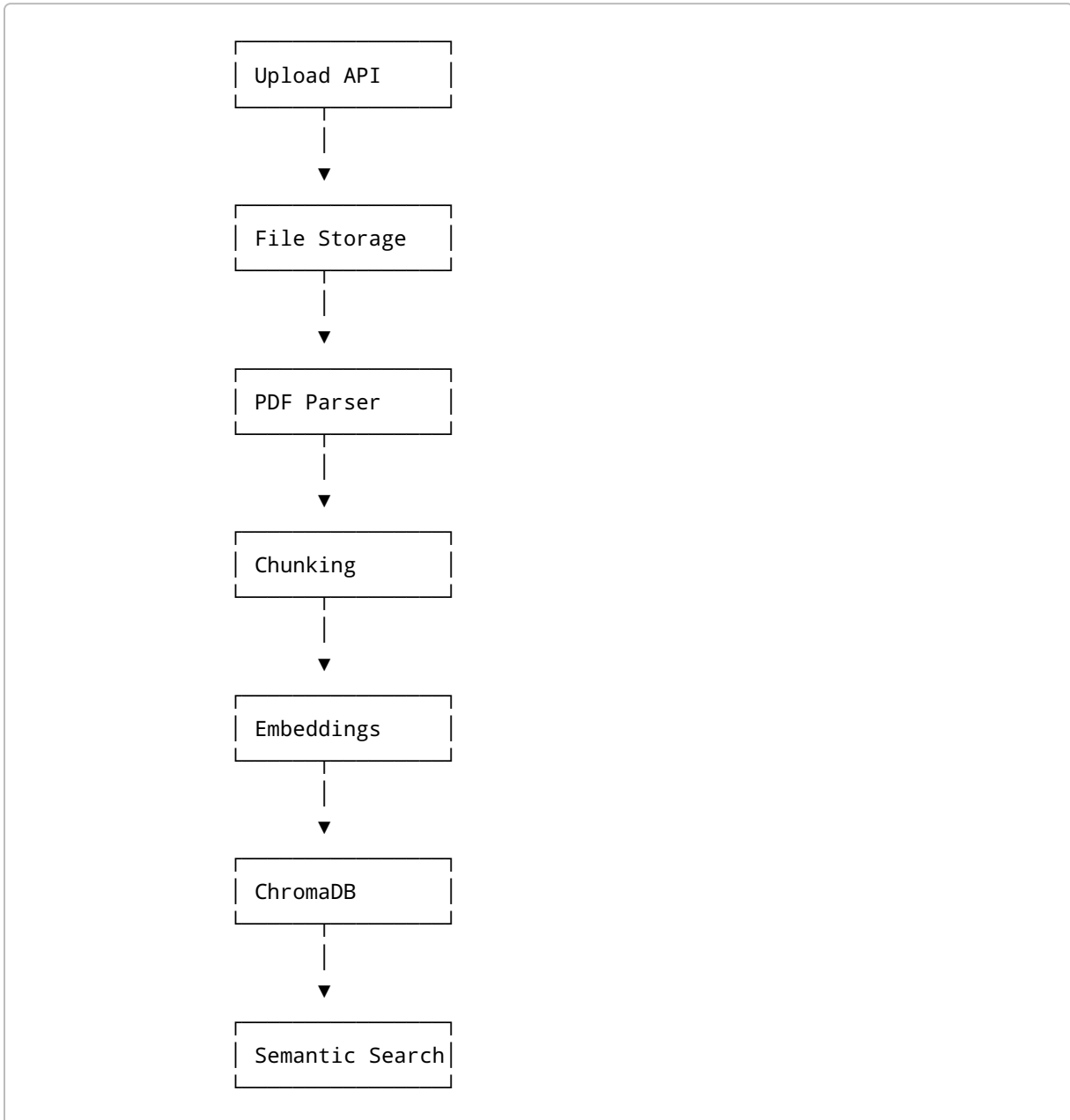
1. PDF Upload
2. File Storage
3. PDF Parsing
4. Chunking
5. Embedding Generation
6. Vector Storage
7. Semantic Retrieval

Excluded in this phase:

- Authentication
- Analytics
- WebSockets

- Mock Tests
 - Adaptive Learning
 - AI Grading
 - Kubernetes
 - Cloud Deployment
-

3. Simplified System Architecture



4. Technology Stack

Component	Technology
Backend Framework	FastAPI
PDF Parsing	PyMuPDF
Embedding Model	Sentence Transformers
Embedding Model Name	all-MiniLM-L6-v2
Vector Database	ChromaDB
Programming Language	Python
API Testing	Postman

5. STEP 1 — Upload API

Objective

The Upload API is responsible for receiving PDF files from users and storing them locally.

Endpoint

`POST /upload`

Responsibilities

- Receive PDF file
- Validate file type
- Save file locally
- Generate unique document ID
- Return document metadata

Example Response

```
{  
  "document_id": "doc_123",  
  "filename": "DBMS.pdf",  
  "status": "uploaded"  
}
```

Local Storage Structure

```
/uploads  
  /doc_123  
    DBMS.pdf
```

6. STEP 2 — PDF Parser

Objective

The parser extracts textual content from uploaded PDF documents.

Tool Used

PyMuPDF

Why PyMuPDF?

- Fast
- Lightweight
- Accurate text extraction
- Preserves page structure

Input

```
{  
  "document_id": "doc_123"  
}
```

Output

```
{  
  "pages": [  
    {  
      "page": 1,  
      "text": "DBMS is a database management system..."  
    }  
  ]  
}
```

Parsing Workflow

```
PDF File  
  ↓  
PyMuPDF  
  ↓  
Extract Pages  
  ↓  
Extract Text  
  ↓  
Store Parsed Output
```

Example Parsing Logic

```
import fitz  
  
doc = fitz.open("DBMS.pdf")
```

```
for page in doc:
    text = page.get_text()
```

7. STEP 3 — Chunking Engine

Objective

Large documents cannot directly be used for retrieval.
Therefore, extracted text must be divided into smaller chunks.

Initial Chunking Strategy

Start with simple token-based chunking.

Configuration

```
Chunk Size = 500 tokens
Overlap = 80 tokens
```

Why Overlap Is Important

Overlap preserves context between chunks.

Without overlap:

- Context breaks
 - Retrieval quality decreases
 - Semantic meaning may be lost
-

Initial Workflow

```
Parsed Text
↓
Token Splitter
```

↓
Generate Chunks

Future Improvements

Later versions can support:

- Heading-aware chunking
- Semantic chunking
- Hierarchical chunking

Example Chunk

```
{  
  "chunk_id": "chunk_1",  
  "text": "Normalization is the process of...",  
  "page": 42  
}
```

8. STEP 4 — Embedding Generation

Objective

Convert textual chunks into vector embeddings for semantic search.

Library Used

```
from sentence_transformers import SentenceTransformer
```

Embedding Model

all-MiniLM-L6-v2

Why this model?

- Fast
- Lightweight
- Free
- Good semantic understanding
- Suitable for MVP systems

Workflow

```
Chunk
  ↓
Embedding Model
  ↓
Vector Representation
```

Example Code

```
model = SentenceTransformer('all-MiniLM-L6-v2')

embedding = model.encode(chunk_text)
```

Example Vector

```
{
  "embedding": [0.12, -0.43, 0.87, ...]
}
```


9. STEP 5 — Vector Storage

Objective

Store embeddings inside a vector database for similarity search.

Vector Database

ChromaDB

Why ChromaDB?

- Open source
 - Easy setup
 - Local development friendly
 - Good for MVPs
-

Data Stored

```
{  
  "chunk": "Normalization removes redundancy...",  
  "page": 12,  
  "document_id": "123"  
}
```

Workflow

```
Embedding  
  ↓  
ChromaDB  
  ↓  
Indexed Vector
```

Benefits

- Fast retrieval
 - Semantic similarity search
 - Efficient indexing
-

10. STEP 6 — Retrieval API

Objective

Search relevant chunks using semantic similarity.

Endpoint

```
POST /search
```

Input

```
{  
  "query": "What is normalization?"  
}
```

Workflow

```
User Query  
  ↓  
Generate Query Embedding  
  ↓  
Vector Similarity Search  
  ↓  
Retrieve Relevant Chunks
```

Example Output

```
{
  "results": [
    {
      "chunk": "Normalization removes redundancy...",
      "page": 42
    }
  ]
}
```

11. Complete Pipeline Workflow

```
Upload PDF
  ↓
Store File
  ↓
Parse PDF
  ↓
Extract Text
  ↓
Generate Chunks
  ↓
Create Embeddings
  ↓
Store in ChromaDB
  ↓
Semantic Retrieval
```

12. Why This Architecture Is Correct

This design directly follows the standard RAG ingestion pipeline.

```
PDF Upload
  ↓
OCR / Parsing
  ↓
Cleaning
```

↓
Chunking
↓
Embeddings
↓
Vector DB

This architecture ensures:

- Scalable retrieval
- Better semantic search
- Grounded AI responses
- Reduced hallucinations

13. Advantages of This MVP

Simple

The architecture avoids unnecessary complexity.

Modular

Each component can later become an independent microservice.

Scalable

The system can later integrate:

- Redis
- Pinecone
- Kubernetes
- Distributed workers

Production Ready Foundation

This pipeline becomes the core of:

- Chat with PDFs
- Mock test generation

- AI tutoring
 - Retrieval-Augmented Generation
-

14. Future Enhancements

Future versions can include:

- OCR for scanned PDFs
 - Semantic chunking
 - Hybrid search
 - Metadata extraction
 - Source citation highlighting
 - Multi-document retrieval
 - Agentic RAG
 - Reranking pipelines
-

15. Conclusion

The designed MVP focuses only on the most critical engineering components of the AI Exam Preparation System.

The complete initial workflow consists of:

1. Uploading PDFs
2. Parsing documents
3. Chunking text
4. Generating embeddings
5. Storing vectors
6. Performing semantic retrieval

This architecture forms the foundation for the complete AI-powered educational platform and enables future expansion into advanced RAG and adaptive learning systems.